GraphOne Intelligence Pipeline - Architecture Document

1. Scale Strategy: Collecting 500,000+ Records

The current pipeline demonstrates the pattern at small scale (1,500–1,600 records per entity type) using single-threaded, polite scraping. To reach 500,000+ records without manual intervention, the architecture would evolve as follows:

- **Horizontal worker scaling**: Replace the single-process scripts with a task queue (Celery + Redis, or AWS SQS) where each "job" is one source/query/page. Multiple worker containers pull jobs in parallel. Scaling to 500k records becomes a matter of increasing worker count, not rewriting code.
- **Source fan-out**: Instead of 5 GitHub awesome-lists, the crawler would ingest hundreds of curated lists, Crunchbase-style directories, YC/AngelList exports, and GitHub Search API results (paginated across topics/stars/languages) — each treated as an independent, parallelizable job.
- **Checkpointing**: Each worker writes progress (last page/cursor processed) to a shared store (Redis or a Postgres crawl_state table) so a crashed worker can resume rather than restart.
- **Rate-aware scheduling**: A per-domain token-bucket rate limiter (shared across workers via Redis) ensures we never exceed a source's tolerance, regardless of how many workers are running.

2. Handling 413 (Payload Too Large) and 429 (Rate Limit) Errors

413 — Context Window Overflow:

- Before sending any text to an LLM, content is chunked using a **semantic-density-aware truncation** strategy: strip boilerplate (nav, footer, ads) via

readability extraction first, then cap at a token budget (e.g. 6,000 tokens for a 8k context model), prioritizing the title, first 3 paragraphs, and any structured data (tables, code blocks) over repeated marketing copy.

- If a single document still exceeds the budget after cleaning, it is split into overlapping chunks (e.g. 500-token overlap) and extraction results are merged, with conflicting fields resolved by preferring the chunk with higher keyword density for that field.

429 — Rate Limits:

- Every LLM call is wrapped in a retry decorator with **exponential backoff + jitter**: wait = min(cap, base * 2^attempt) + random(0, 1), capped at ~60s, max 5 retries.
- **Fallback chain**: Gemini Flash (primary, cheapest/fastest) → Groq Llama 3 (secondary) → DeepSeek (tertiary). On a 429 or repeated timeout from one provider, the orchestrator immediately tries the next tier rather than waiting out the full backoff on a saturated provider — this keeps throughput high across thousands of concurrent extractions since load spreads across providers.
- Concurrency is capped per-provider (semaphore-limited async workers) so we proactively stay under each provider's published rate limit rather than reactively backing off after every 429.

3. Freshness Tracking Across Distributed Nodes

- Every ingested record is keyed by a **content hash** (SHA-256 of normalized URL + title) and written to a shared deduplication store (Redis Set or a Postgres unique index on content_hash).
- Before processing any article/job, a worker checks this store; a hit means "already processed," regardless of which node originally fetched it — this makes deduplication safe across any number of distributed crawler nodes.
- For sources without reliable timestamps, a **heuristic freshness check** compares the content hash against the last-seen hash for that source+category combination; if unseen, it's treated as new and passed

through a lightweight recency classifier (checks for relative-date phrases, "just now," etc.) as a secondary signal.

- All timestamps are normalized to UTC ISO-8601 using dateutil, handling absolute (RFC822, ISO), and relative ("2 hours ago") formats consistently, so the 24-hour cutoff check is timezone-safe across sources.

4. Storage Strategy

- **Primary store — PostgreSQL**: Chosen for the structured entity records (Startups, Products, Papers, Jobs, News) because the schema is well-defined, relationships (startup → product → job postings) are relational by nature, and Postgres's JSONB columns let us keep flexible content.data fields without sacrificing queryability or ACID guarantees needed for deduplication.
- **Vector store — pgvector (or a dedicated store like Pinecone/Weaviate) for entity embeddings**: Used specifically for **fuzzy entity resolution at scale** — instead of pairwise fuzzy-string matching (which doesn't scale past tens of thousands of records), canonical entity names are embedded once, and new raw names are matched via nearest-neighbor vector search, which stays fast even at 500k+ entities.
- **Graph store — Neo4j (or Postgres with a graph-modeling layer)**: Used to map the *relationships* the task emphasizes — founder→startup, startup→product, paper→GitHub repo→startup (if the repo maps to a company), and job→company. This is what lets the "Intelligence Graph" support queries like "show me all products from startups founded by ex-DeepMind researchers," which a flat relational table can't do efficiently.
- This three-tier approach (relational for facts, vector for fuzzy matching, graph for relationships) mirrors how production knowledge-graph systems are typically built, and each store scales independently as data volume grows.

5. Anti-Bot & Compliance Strategy (Current Implementation)

For this trial, all sources were deliberately chosen to be **API-first or RSS-first** (arXiv API, GitHub API, RSS feeds, raw GitHub content) rather than scraping Cloudflare/Datadome-protected pages directly. This was a conscious tradeoff: it guarantees 100% real, traceable data with zero risk of captcha-triggering or IP bans, at the cost of not demonstrating a live headless-browser bypass.

For production-scale ingestion of protected sources (e.g. Crunchbase, LinkedIn):

- **Playwright (async) with residential proxy rotation** to render JS-heavy pages and rotate IP/fingerprint per request.
- **Managed unblocking services** (e.g. Bright Data, ScraperAPI) for consistently protected domains, since building and maintaining custom captcha-solving infrastructure is rarely worth it compared to a managed service at this scale.
- **Respect robots.txt and rate limits** even when technically bypassable, to keep the pipeline sustainable and reduce legal/ethical risk — this was a explicit design principle throughout this build.